CENTRO UNIVERSITÁRIO BELAS ARTES

Jogo de Bingo em Java: Arquitetura Observer e Otimização com Lean

MANUELA SANTOS LOBO / RM: 24109798

São Paulo
2025

Introdução

Este trabalho envolve o desenvolvimento de um jogo básico de Bingo, empregando a linguagem Java e uma interface visual com Swing. A meta principal é replicar o processo de seleção de números e o preenchimento de um cartão, aplicando o padrão de design Observer para alertar o cartão sempre que um número é sorteado.

Descrição do Problema

O Bingo é um jogo de azar que consiste no sorteio aleatório de dígitos de 1 a 75 (ou 90 em variantes diferentes) e na conferência para saber se
os participantes têm esses dígitos em suas folhas de jogo. Vence quem completar todos

3. Implementação

3.1 Classes Principais

- Sorteador: Uma classe encarregada de conduzir o sorteio dos números. Esta classe mantém uma lista embaralhada dos números disponíveis e avisa os observadores sempre que um número é sorteado. Ela implementa a classe Observable.

Main.java | Sorteador.java | CartelaBingo.java

C:\Users\Manuela Lobo\IdeaProjects\Jogo do Bingo\src\Sorteador.java

```java
import java.

public class Sorteador extends Observable {  2 usages
    private final List<Integer> numerosDisponiveis;  5 usages

    public Sorteador() {  1 usage
        numerosDisponiveis = new ArrayList<>();
        for (int i = 1; i <= 75; i++) {
            numerosDisponiveis.add(i);
        }
        Collections.shuffle(numerosDisponiveis);
    }

    public int sortearNumero() {  1 usage
        if (!numerosDisponiveis.isEmpty()) {
            int numero = numerosDisponiveis.remove( index: 0);
            setChanged();
            notifyObservers(numero);
            return numero;
        }
        return -1;
    }
}
```

Main.java Sorteador.java CartelaBingo.java

```java
import javax.swing.*;
import java.awt.*;
import java.util.*;

public class CartelaBingo extends JPanel implements Observer {  2 usages
    private final Set<Integer> numerosCartela;  4 usages
    private final Set<Integer> numerosMarcados;  4 usages

    public CartelaBingo() {  1 usage
        this.numerosCartela = gerarCartela();
        this.numerosMarcados = new HashSet<>();
        setPreferredSize(new Dimension( width: 400,  height: 400));
    }

    private Set<Integer> gerarCartela() {  1 usage
        Set<Integer> cartela = new LinkedHashSet<>();
        Random random = new Random();
        while (cartela.size() < 24) {
            int numero = random.nextInt( bound: 75) + 1;
            cartela.add(numero);
        }
        return cartela;
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        int x = 20, y = 30;
        int count = 0;

        for (int numero : numerosCartela) {
            if (numerosMarcados.contains(numero)) {
```

CartelaBingo: Esta classe representa a cartela do jogador, contendo 24 números escolhidos de forma aleatória. Ela implementa a interface Observer, que recebe avisos do Sorteador e marca os números que foram sorteados, atualizando a interface visual para exibir os números que foram marcados.

Main: Classe responsável por criar a interface visual, configurar a cartela e o sorteador, além de fornecer um botão para iniciar o sorteio. Ela gerencia a interação do usuário com o jogo.

Main.java | Sorteador.java | CartelaBingo.java

```java
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class Main extends JFrame {
    private CartelaBingo cartela;  6 usages
    private Sorteador sorteador;  3 usages
    private JButton botaoSortear;  5 usages
    private JLabel labelNumeroSorteado;  4 usages

    public Main() {  2 usages
        super( title: "Jogo do Bingo");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize( width: 400,  height: 500);
        setLayout(new BorderLayout());

        cartela = new CartelaBingo();
        sorteador = new Sorteador();

        sorteador.addObserver(cartela);

        botaoSortear = new JButton( text: "Sortear Número");
        labelNumeroSorteado = new JLabel( text: "Número sorteado: ", SwingConstants.CENTER);

        botaoSortear.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                if (!cartela.estaCompleta()) {
                    int numero = sorteador.sortearNumero();
                    if (numero != -1) {
                        labelNumeroSorteado.setText("Número sorteado: " + numero);
```

3.2 Funcionamento

- Ao começar o programa, uma cartela é criada com 24 dígitos distintos variando de 1 a 75.
- usuário aperta o botão "Sortear Número", que invoca o método do sorteador para gerar um número aleatório.
- sorteador avisa a cartela através do padrão Observer para que o número seja marcado, caso esteja presente.
- A cartela é atualizada para destacar os números que foram marcados.
- Quando todos os dígitos da cartela estiverem assinalados, o programa apresenta uma mensagem de "BINGO" e desativa o botão de sorteio.

4. Tecnologias Utilizadas

1- Linguagem de programação: Java
2- Biblioteca gráfica: Swing
3- Padrão de projeto: Observer

5. Padrão Observer no Projeto

O padrão Observer foi empregado para separar o gerador de números da cartela. O gerador (Observable) não precisa saber pormenores da cartela, apenas informa os seus observadores sobre o número que foi sorteado. A cartela (Observer) reage a essa informação atualizando tanto seu estado quanto a interface do usuário.

6. Roadmap de evolução do produto

Mês 1 – Planejamento Inicial

- Definição dos requisitos principais do jogo (cartela, sorteio, marcação).
- Escolha da linguagem e padrão de arquitetura (Observer).
- Protótipo simples da interface.

Mês 2 – Construção do MVP

- Desenvolvimento do núcleo do sistema: mecanismo de sorteio e cartela.
- Implementação da primeira versão da interface gráfica.
- Integração do padrão Observer para conectar sorteio → cartela.

Mês 3 – Testes e Estabilidade

- Testes do sorteio e conferência da cartela.
- Correções de bug e comportamento irregular.
- Ajustes na renderização visual (marcação dos números).

Mês 4 – Melhorias de Experiência

- Interface mais amigável e organizada.
- Destaque visual para números marcados.
- Mensagens de feedback ao usuário (ex: Bingo, erro, repetição).

Mês 5 – Expansão de Funcionalidades

- Suporte para múltiplas cartelas.
- Modo “simulação automática”.
- Registro básico das jogadas.

Mês 6 – Evolução e Preparação para Escalar

- Modularização completa do projeto.
- Possível migração futura para JavaFX ou web.
- Documentação final do software.

7. Aplicação do Lean no Desenvolvimento

O Lean pode ser aplicado no projeto para tornar o desenvolvimento mais rápido, eficiente e sem desperdícios. Aqui estão os pontos principais:

1. Eliminação de Desperdícios

- Focar apenas no essencial do MVP (cartela, sorteio, marcação).
- Evitar criar recursos desnecessários antes da hora.

2. Entregas Rápidas e Incrementais

- Dividir o desenvolvimento em pequenas entregas, testando cada parte assim que estiver pronta.
- Liberar primeiro a versão mínima funcional e ir aprimorando.

3. Aprendizado Contínuo

- Ajustar rapidamente o projeto conforme erros são encontrados.
- Coletar feedback sobre a interface e funcionamento do jogo.

4. Qualidade Integrada desde o Início

- Uso do padrão Observer para reduzir acoplamento e facilitar manutenção.
- Código organizado em camadas simples e claras.

5. Redução da Complexidade

- Interface inicial simples, evitando sobrecarregar o jogador com elementos não essenciais.
- Classes com funções bem definidas para manter o projeto limpo.

Conclusão

O desenvolvimento do jogo de Bingo, utilizando Java e Swing, proporcionou uma aplicação prática e didática de conceitos fundamentais de programação orientada a objetos, especialmente o padrão Observer. A separação entre o sorteador (Observable) e a cartela (Observer) evidenciou a eficiência deste padrão ao manter um baixo acoplamento, além de facilitar a manutenção e evolução do sistema. A organização do projeto, aliada a um roadmap bem estruturado e à utilização dos princípios Lean, permitiu entregas incrementais, testes contínuos e melhorias graduais na experiência do usuário.

O projeto também destacou a importância de iniciar com o mínimo viável, priorizando uma funcionalidade básica antes de avançar para expansões como múltiplas cartelas ou simulação automática. Dessa forma, foi possível desenvolver um jogo funcional, intuitivo e com alto potencial de escalabilidade, preparado para evoluir em versões mais complexas ou até mesmo para outras plataformas.

Assim, a integração de boas práticas de desenvolvimento, padrões de projeto e metodologias ágeis culminou em um aprendizado significativo e na criação de um produto final bem estruturado, funcional e com perspectivas claras de expansão futura.

Referências

Oracle. *The Java™ Tutorials – Learning Swing*. Disponível em: https://docs.oracle.com/javase/tutorial/uiswing/

GeeksforGeeks. *Java Swing Tutorial*. Disponível em: https://www.geeksforgeeks.org/java/introduction-to-java-swing/

Baeldung. *Observer Design Pattern in Java*. Disponível em: https://www.baeldung.com/java-observer-pattern

ZetCode. *Java Swing Tutorial*. Disponível em: https://zetcode.com/javaswing/

Lean.org. *What is Lean?*. Disponível em: https://www.lean.org/whatslean/